Name: Logesh P	Register / Roll No: 192521351
Course Code: DSA0113	Assignment: 01

TASK 1 – Understand the Problem and Identify the Objects

1. Objects in the system

An object is an instance of a class. The main objects in the metro fare system are:

Object	Data (attributes)	Behaviour (methods)
SmartCard	Card number, balance	Check balance, deduct fare, recharge
Passenger	Passenger ID, passenger type	Select journey, receive concession
Journey	Source, destination, time, zone	Calculate journey fare
Zone	Zone name, base fare	Provide zone fare
FareTable	Zone fares, surcharge, limits	Return correct fare

Explanation:

1. SmartCard: Stores the passenger's money. It checks whether enough balance is available before allowing a journey.
2. Passenger: Represents the person travelling. It stores passenger details and concession eligibility.
3. Journey: Represents one metro trip. It contains the travel zone and journey time.
4. Zone: Represents Inner, Middle or Outer zone and its fare.

5. FareTable: Stores all fare rules in one place. This avoids typing fare values repeatedly in the program.

2. Procedural programming vs OOP

Procedural programming:

- Uses separate variables and functions.
- Data and functions are maintained separately.
- Example: balance, fare, calculateFare() and deductFare().

Object-oriented programming:

- Uses classes and objects.
- Combines data and functions.
- Example: A SmartCard class stores the balance and provides a deduct() method.

OOP is better for this project because each object has a separate responsibility. It is easier to maintain and update.

3. Four basic OOP principles

1. Encapsulation: Keeps the smart-card balance private. Other parts of the program cannot directly change it.
2. Abstraction: The user calls deduct() without knowing all the internal balance-checking steps.
3. Inheritance: A StudentPassenger and SeniorPassenger class can inherit common properties from a Passenger class.
4. Polymorphism: Different passenger types can use the same calculateFare() function but apply different concession rules.

4. Rules for a correct journey

- The selected zone must be valid.
- The fare must be calculated correctly.
- Peak surcharge must apply only during the specified peak hours.
- Money must be stored in integer paise to avoid rounding errors.
- The balance after deduction must be at least Rs 50.
- A negative recharge must not be accepted.
- An invalid journey must not deduct money.

5. Alternative concept and complexity

If we use procedural programming, we can store the balance in a variable and use separate functions.

```
long long balance = 10000;

bool canTravel(long long fare)
{
   return balance - fare >= 5000;
}

void deductFare(long long fare)
{
   if (canTravel(fare))
      balance -= fare;
}
```

Comparison	Procedural	OOP
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)
Data protection	Less	Better
Maintenance	More difficult	Easier

Conclusion: OOP is recommended because it protects card data and makes future fare changes easier. Both approaches have the same Big-O time and space complexity for one journey.

Code:

```
#include <iostream>

#include <string>

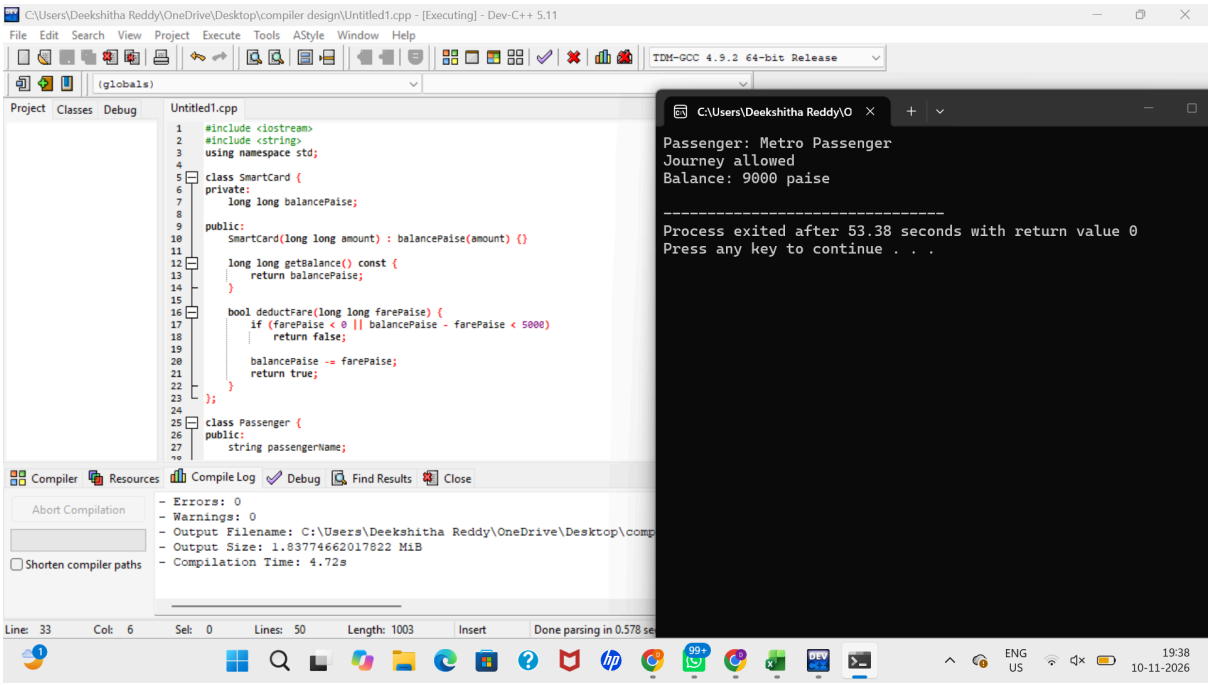
using namespace std;

class SmartCard {

private:
```

```
    long long balancePaise;
public:
    SmartCard(long long amount) : balancePaise(amount) {}
    long long getBalance() const {
        return balancePaise;
    }
    bool deductFare(long long farePaise) {
        if (farePaise < 0 || balancePaise - farePaise < 5000)
            return false;
        balancePaise -= farePaise;
        return true;
    }
};
class Passenger {
public:
    string passengerName;
    Passenger(string name) : passengerName(name) {}
    void display() const {
        cout << "Passenger: " << passengerName << '\n';
    }
};
int main() {
    Passenger passenger("Metro Passenger");
    SmartCard card(10000);
```

```
    passenger.display();
    if (card.deductFare(1000))
        cout << "Journey allowed\n";
    else
        cout << "Journey denied\n";
    cout << "Balance: " << card.getBalance() << " paise\n";
    return 0;
}
```



TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

1. Five types of C++ tokens

Tokens are the smallest individual units in a C++ program.

Token	Meaning	Example
Keywords	Reserved words	int, class, return
Identifiers	Programmer-defined names	balance, fare
Constants	Fixed values	10, 5000, 20
Operators	Perform operations	+, -, ==
Punctuators	Separate program elements	{}, ;, ()

2. Difference between #define, const and constexpr

- #define is a preprocessor macro. It does not provide normal C++ type checking.
- const creates a value that cannot be changed after initialization.
- constexpr declares a value that can be evaluated at compile time when its initializer permits it.

For the fare system, constexpr is suitable because the fare values are fixed during compilation.

onstexpr int INNER_FARE = 1000;
constexpr int MIDDLE_FARE = 2000;
constexpr int OUTER_FARE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE = 5000;
The values are stored in paise.

3. enum and enum class

An enum represents a fixed group of named values.

```
enum Zone

{
   Inner,
   Middle,
   Outer
};
```

A safer alternative is:

```
enum class Zone
{
   Inner,
   Middle,
   Outer
};
```

Why enum class is better:

- It provides better type safety.
- Its values belong to their own scope.
- It avoids accidental conversion to integers.
- It prevents naming conflicts.

4. Naming rules

- Variable names should begin with a letter or underscore.
- Names cannot contain spaces.
- C++ keywords cannot be used as identifiers.
- Names should be meaningful.
- Constants are commonly written in uppercase.

5. Alternative concept: using macros

```
#define INNER_FARE 1000
#define MIDDLE_FARE 2000
#define OUTER_FARE 3000
```

Macros are replaced by the preprocessor before compilation. They do not provide the same type checking and scope rules as C++ constants.

Feature	Macro	constexpr

Type checking	No normal type checking	Yes
Scope	Preprocessor-based	C++ scope
Memory	Usually no storage of its own	Usually no storage of its own
Running time	No extra runtime lookup	No extra runtime lookup

Conclusion: constexpr and enum class are recommended because they make the program safer and easier to understand.

Code:

```
#include <iostream>

using namespace std;

constexpr int INNER_FARE = 1000;

constexpr int MIDDLE_FARE = 2000;

constexpr int OUTER_FARE = 3000;

constexpr int PEAK_PERCENT = 20;

enum class Zone {

    Inner,

    Middle,

    Outer

};

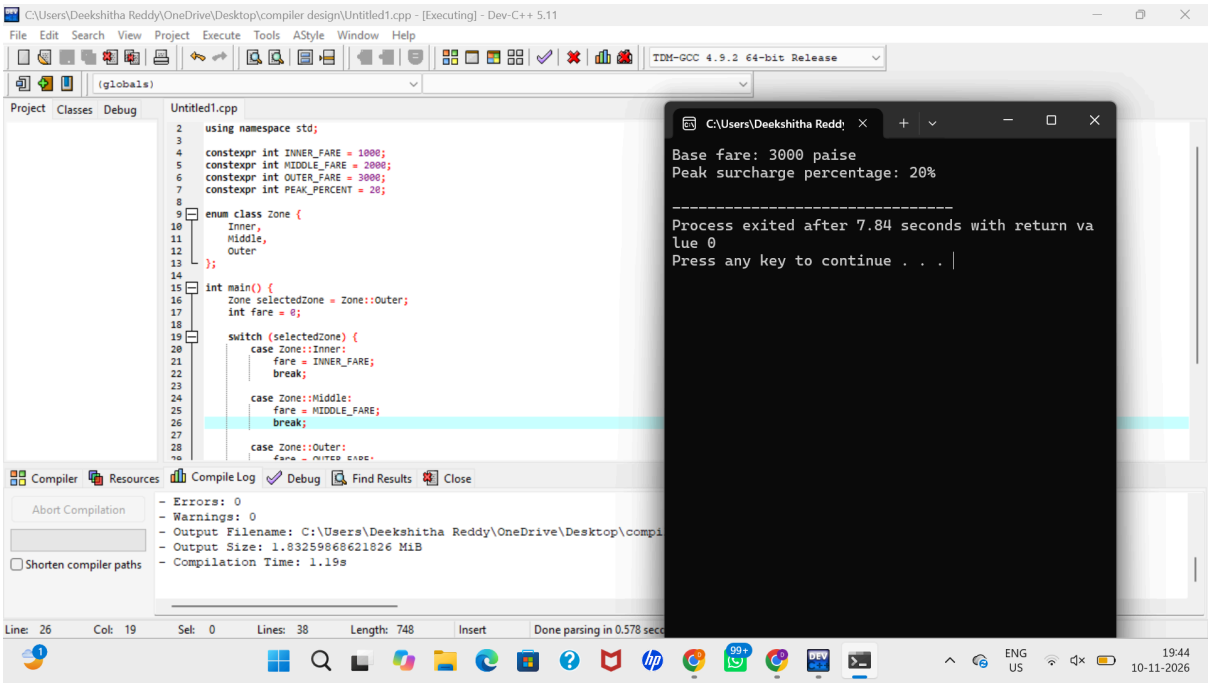
int main() {

    Zone selectedZone = Zone::Outer;

    int fare = 0;

    switch (selectedZone) {
```

```
        case Zone::Inner:
            fare = INNER_FARE;
            break;
        case Zone::Middle:
            fare = MIDDLE_FARE;
            break;
        case Zone::Outer:
            fare = OUTER_FARE;
            break;
    }
    cout << "Base fare: " << fare << " paise\n";
    cout << "Peak surcharge percentage: "
         << PEAK_PERCENT << "%\n";
    return 0;
}
```



TASK 3 – Choose Data Types and Check Type Compatibility

1. Suitable data types

Value	Data type	Reason
Fare in paise	long long	Stores exact money values
Card balance	long long	Supports large balances
Journey count	long long	Supports millions of journeys
Hour	int	Stores values from 0 to 23
Zone	enum class	Represents fixed zones
Passenger eligibility	bool	Stores true or false
Passenger initial	char	Stores one character
Average fare	double	Useful for calculations and display
Fare percentage	int	Stores whole-number percentage

2. Why use integer paise instead of double rupees?

For example:

- Rs 10 = 1000 paise.
- Rs 20 = 2000 paise.
- Rs 30 = 3000 paise.

Floating-point numbers may introduce small representation errors. Integer paise stores money exactly in the whole paise.

long long fare = 1200; // Rs 12.00

3. Type compatibility

C++ converts values when different data types are used in an expression.

Example:

int a = 10;

double b = 2.5;

double result = a + b;

Here, a is converted from int to double. The result is 12.5.

Example with Boolean:

int x = 5;

bool status = x;

Since x is non-zero, status becomes true.

4. Size and range

The exact sizes depend on the compiler and system. The following is common on a 64-bit system:

Type	Common size	Common range
bool	1 byte	true or false
char	1 byte	Usually -128 to 127 or 0 to 255
int	4 bytes	-2,147,483,648 to 2,147,483,647

unsigned int	4 bytes	0 to 4,294,967,295
long long	8 bytes	At least -9.22 × 10¹?? to 9.22 × 10¹??
float	4 bytes	Approximately 6?7 decimal digits
double	8 bytes	Approximately 15?16 decimal digits

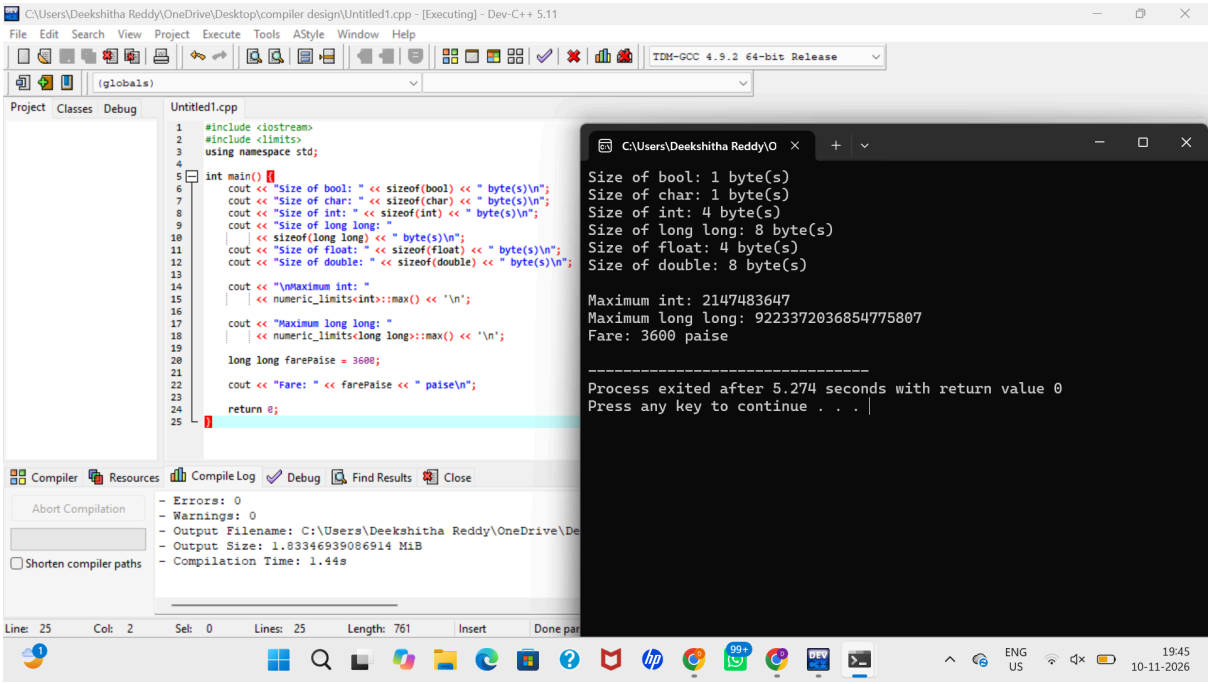
5. Program to check sizes

```
#include <iostream>
using namespace std;

int main()
{
    cout << "int: " << sizeof(int) << " bytes\n";
    cout << "long long: " << sizeof(long long) << " bytes\n";
    cout << "double: " << sizeof(double) << " bytes\n";
    cout << "float: " << sizeof(float) << " bytes\n";
    cout << "bool: " << sizeof(bool) << " bytes\n";
    cout << "char: " << sizeof(char) << " bytes\n";

    return 0;
```

}



6. Is int enough for network collection?

Assume each journey costs Rs 36, or 3600 paise.

Journeys per day	Total in paise	Total in rupees
20,000	72,000,000	Rs 7,20,000
500,000	1,800,000,000	Rs 1,80,00,000
3,000,000	10,800,000,000	Rs 10,80,00,000

The maximum positive int is 2,147,483,647. Therefore, the whole-network total of 10,800,000,000 paise cannot fit in an ordinary 32-bit int.

Conclusion: Use long long for money totals and journey counts.

7. Alternative: using double

double total = 0.0;

```
for (int i = 0; i < 1000; i++)
{
   total += 10.20;
}
```

The result may contain a small floating-point representation error.

Both double and long long commonly occupy 8 bytes. Both take O(1) time per addition. However, integer paise is more reliable for money.

TASK 4 – Control Implicit Conversions

1. What are implicit conversions?

Implicit conversion means C++ automatically changes one data type into another.

Example:

int a = 10;

double b = a;

Here, int is automatically converted to double.

2. Integer division

int a = 7;
int b = 2;

double result = a / b;

The result is 3, not 3.5, because integer division happens first.

Correct method:

double result = static_cast<double>(a) / b;

The result is 3.5.

3. Signed and unsigned comparison

int s = -1;
unsigned int u = 1;

When these are compared, the signed value may be converted to unsigned. Therefore, s < u is false on the usual systems because the converted value of s is a large unsigned value.

This comparison can produce a compiler warning.

4. Narrowing conversion

int x = 3.7;

The decimal part is lost, so x becomes 3.

Brace initialization prevents this narrowing:

// int x{3.7}; // Compilation error

5. Use of static_cast

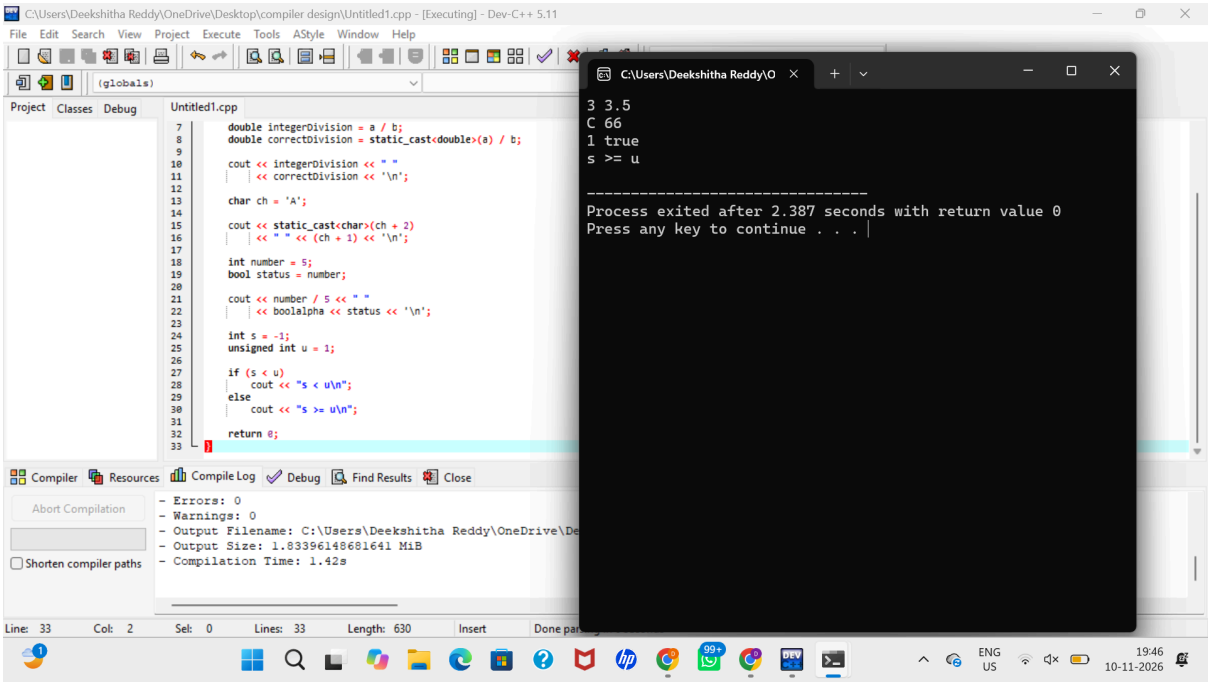
static_cast performs an explicit conversion.

int a = 7;

double b = static_cast<double>(a);

It is useful when we want to clearly control conversions.

6. Predict the output



Explanation:

1. a / b performs integer division, so 7 / 2 gives 3. The second expression converts a to double, so it gives 3.5.
2. 'A' + 2 produces the character C. In ch + 1, the character is promoted to an integer, giving 68.
3. Integer 5 converts to Boolean true. The first output prints 1; boolalpha prints true.
4. The signed value -1 is converted to unsigned during comparison. Therefore, the condition s < u is false.

7. Alternative concept: explicit conversions

Instead of depending on implicit conversions, we can use:

int a{7};

int b{2};

double result = static_cast<double>(a) / b;

This improves clarity and helps prevent accidental conversions. It does not normally change the running time or memory use.

TASK 5 – Write Correct Expressions with the Right Precedence

1. Peak-hour condition

```
bool peak =
   (hour >= 8 && hour < 10) ||
   (hour >= 17 && hour < 20);
```

The condition covers 8:00–9:59 AM and 5:00–7:59 PM.

At exactly 10 AM and 8 PM, the condition is false.

2. Surcharge calculation

```
fare = fare + (fare * 20 / 100);
```

For Rs 30:

- Base fare = 3000 paise.
- Surcharge = 3000 × 20 / 100 = 600 paise.
- Final fare = 3600 paise = Rs 36.

3. Concession expression

The assignment does not give a specific concession percentage. If the metro defines a 10% concession, for example:

```
fare = fare - (fare * 10 / 100);
```

This is only an example percentage; use the actual percentage if your faculty provides one.

4. Operator precedence

Operator precedence decides which operation is performed first.

Example:

```
int result = 5 + 3 * 2;
```

Multiplication happens first, so the result is 11.

Using brackets:

```
int result = (5 + 3) * 2;
```

The result is 16.

5. Difference between x++ and ++x

- x++: Uses the old value first, then increments.
- ++x: Increments first, then uses the new value.

Example:

int x = 5;

cout << x++;

Output: 5.

int y = 5;

cout << ++y;

Output: 6.

i = i++ is a bad expression because it is confusing and should be avoided. In C++17 and later, this expression assigns the old value back, so the increment is not retained.

6. Division and modulus with negative numbers

cout << -10 / 3;
cout << -10 % 3;

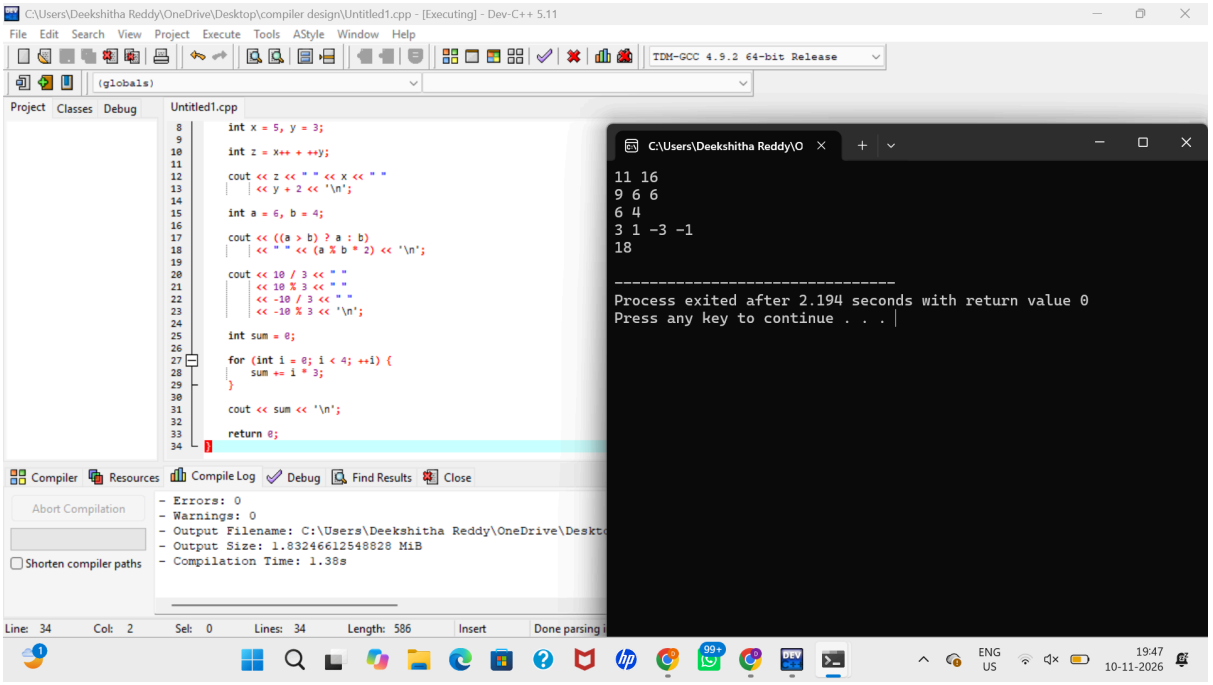
Output:

-3

-1

C++ integer division truncates toward zero. The remainder has the sign of the left operand.

7. Predict the output



Explanation:

- First line: 3 * 2 = 6, so 5 + 6 = 11; (5 + 3) * 2 = 16.
- Second line: x++ uses 5 and then increments x to 6. ++y increments 3 to 4 before using it. Therefore, z = 5 + 4 = 9.
- Third line: x is 6 and y is 4. The conditional expression returns 6. Also, 6 % 4 * 2 = 2 * 2 = 4.
- Fourth line: Integer division and modulus give 3, 1, -3, and -1.
- Fifth line: The loop adds 0 + 3 + 6 + 9 = 18.

8. Alternative concept: lookup table

Instead of using a combined condition, we can store peak hours in an array.

```
bool peak[24] = {};


for (int i = 8; i < 10; i++)
    peak[i] = true;


for (int i = 17; i < 20; i++)
    peak[i] = true;
```

Then:

```
bool isPeak = peak[hour];
```

Both methods use O(1) time per check. The lookup table requires additional space for 24 Boolean values, but it can make repeated checks simple.

TASK 6 – Design the Control Structures

1. Use of control structures

Control structure	Purpose
if-else	Check peak hours and card balance
switch	Select the fare based on the zone
Conditional operator ?:	Select between two values
for loop	Process a fixed number of journeys
Range-based for	Process every journey in an array
while loop	Continue until the user chooses to stop
break	Exit a loop or switch
continue	Skip an invalid journey

Best choice: A range-based for loop is suitable when the journeys are already stored in an array or collection. A while loop is suitable for continuous gate operation.

2. Pseudocode

START

SET Inner fare = 1000 paise
SET Middle fare = 2000 paise
SET Outer fare = 3000 paise
SET Minimum balance = 5000 paise

```
READ zone
READ hour
READ card balance

IF zone is invalid
   DISPLAY "Invalid zone"
   STOP
END IF

IF hour is invalid
   DISPLAY "Invalid hour"
   STOP
END IF

USE SWITCH to select base fare

IF hour is between 8 and 10
   OR hour is between 17 and 20
   ADD 20% surcharge
END IF

IF balance - fare is at least 5000 paise
   DEDUCT fare
   DISPLAY fare and balance
ELSE
   DISPLAY "Journey denied"
END IF

STOP
```

3. Complete the code

The following program completes TODO 1 and TODO 2 using a switch statement and integer paise.

```
#include <iomanip>

#include <iostream>

using namespace std;



enum class Zone { Inner, Middle, Outer };
```

```
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base = 0;

    switch (z)
    {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;

        case Zone::Middle:
            base = BASE_PAISE[1];
            break;

        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return -1;
```

```
    }

    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;

    return base;
}

bool canTravel(long long balance, long long f)
{
    return f >= 0 &&
           balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000;

    struct Trip
    {
        Zone zone;
        bool peak;
    };

    Trip trips[] = {
```

```
    {Zone::Inner, false},
    {Zone::Outer, true},
    {Zone::Middle, false}
};

for (const Trip& t : trips)
{
    long long f = fare(t.zone, t.peak);

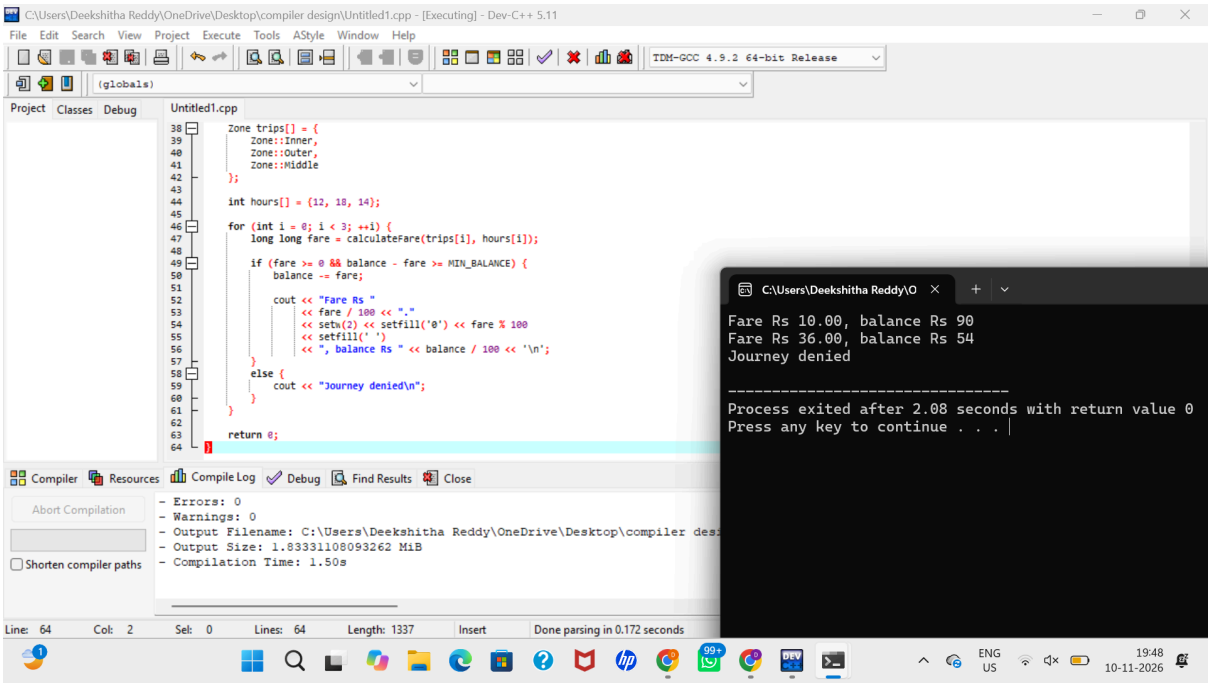
    if (canTravel(balance, f))
    {
        balance -= f;

        cout << "Fare Rs "
             << f / 100 << '.'
             << setw(2) << setfill('0')
             << f % 100 << setfill(' ');
    }
    else
    {
        cout << "Denied (low balance)";
    }

    cout << " balance Rs " << balance / 100 << '\n';
}
```

```
    return 0;
}
```

Expected output:



4. Add boundary-time checks

The peak condition must be:

```
bool peak =
    (hour >= 8 && hour < 10) ||
    (hour >= 17 && hour < 20);
```

Hour	Peak?
8	Yes
9	Yes

10	No
17	Yes
19	Yes
20	No

5. Alternative concept: array instead of switch

constexpr long long FARE[] = {1000, 2000, 3000};

A valid zone index can be used to access the array.

Both switch and array lookup are O(1) for a fixed number of zones. An array is easier to extend when more zones are added, but the index must be validated.

TASK 7 – From Procedural to Object-Oriented

1. Procedural implementation

In procedural programming, the balance is stored separately and functions perform the operations.

```
#include <iostream>

using namespace std;


long long balance = 10000;

const long long MIN_BALANCE = 5000;


bool canTravel(long long fare)

{

   return balance - fare >= MIN_BALANCE;

}


void deduct(long long fare)

{

   if (canTravel(fare))

   {

      balance -= fare;

      cout << "Journey allowed\n";

   }

   else

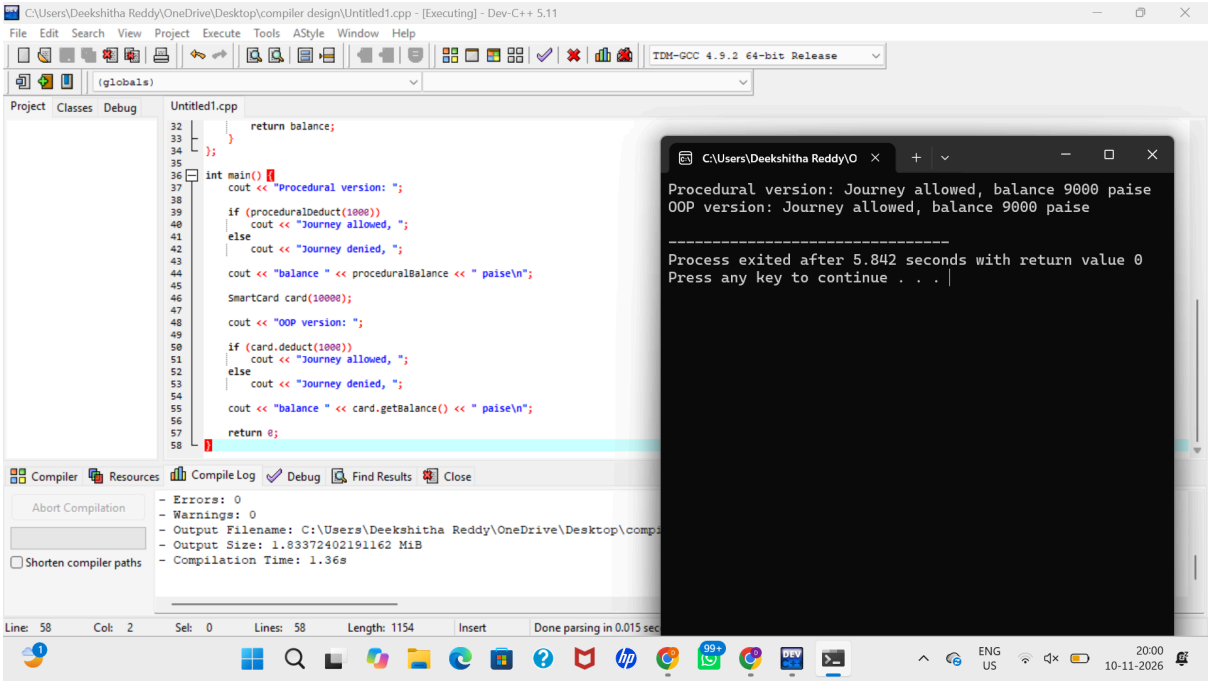
   {

      cout << "Journey denied\n";
```

```
    }
}

int main()
{
    deduct(1000);
    cout << "Balance: " << balance << " paise\n";

    return 0;
}
```



2. Object-oriented implementation

In OOP, the balance is kept private inside the SmartCard class.

```
#include <iostream>

using namespace std;

class SmartCard
```

```
{
private:
    long long balance;
    const long long MIN_BALANCE = 5000;

public:
    SmartCard(long long amount)
    {
        balance = amount;
    }

    bool canTravel(long long fare)
    {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare)
    {
        if (canTravel(fare))
        {
            balance -= fare;
            cout << "Journey allowed\n";
        }
        else
```

```
        {
            cout << "Journey denied\n";
        }
    }

    long long getBalance()
    {
        return balance;
    }
};

int main()
{
    SmartCard card(10000);

    card.deduct(1000);

    cout << "Balance: "
         << card.getBalance() << " paise\n";

    return 0;
}
```

3. Comparison

Feature	Procedural	OOP

Data	Separate variables	Private class members
Functions	Free functions	Class methods
Data protection	Less	Better
Maintenance	More difficult	Easier
Time per journey	O(1)	O(1)
Space per card	O(1)	O(1)

4. How does OOP prevent a negative balance?

The balance is private. It can only be changed through the deduct() method. The method checks whether enough money remains before deducting the fare.

5. Alternative concept: public balance

If the balance were public:

card.balance = -500;

Any part of the program could change it directly. This could break the minimum-balance rule.

Making the balance private does not change the Big-O time or space complexity. It improves safety and control.

Conclusion: The OOP version is recommended because it protects the balance and makes changes easier.

TASK 8 – Read and Print Data Safely

1. Reading input using cin

The program should check whether the user entered a valid number.

int hour;

cout << "Enter hour: ";

cin >> hour;

if (cin.fail())

{

cout << "Invalid input";

}

2. Recovering from invalid input

When the user enters letters instead of numbers, cin enters a failure state.

We can recover using:

- cin.clear() – clears the error state.
- cin.ignore() – removes the invalid input.

3. Safe input program

```
#include <iostream>
#include <limits>
using namespace std;

int main()
{
   int hour;

   while (true)
   {
      cout << "Enter hour (0-23): ";
      cin >> hour;
```

```
        if (cin.fail())
        {
            cout << "Invalid input. Enter numbers only.\n";

            cin.clear();
            cin.ignore(
                numeric_limits<streamsize>::max(), '\n'
            );
            continue;
        }

        if (hour < 0 || hour > 23)
        {
            cout << "Hour must be between 0 and 23.\n";
            continue;
        }

        break;
    }

    cout << "Valid hour: " << hour << '\n';

    return 0;
}
```

4. Formatting money using <iomanip>

Useful formatting functions:

- setw() – sets the output width.
- setfill() – fills empty spaces with a character.
- fixed – displays fixed decimal places.
- setprecision() – controls decimal places.

Example:

```
#include <iostream>

#include <iomanip>

using namespace std;



int main()
```

```
{
    double fare = 36.0;


    cout << fixed << setprecision(2);
    cout << "Fare: Rs " << fare << '\n';


    return 0;
}
```

Output:

Fare: Rs 36.00

For actual fare calculations, continue to store money in integer paise and format it when displaying.

5. Alternative concept: getline() and stoi()

We can read the entire input line and convert it into an integer.

```
#include <iostream>
#include <string>
using namespace std;


int main()
{
    string input;
    int hour;


    cout << "Enter hour: ";
    getline(cin, input);
```

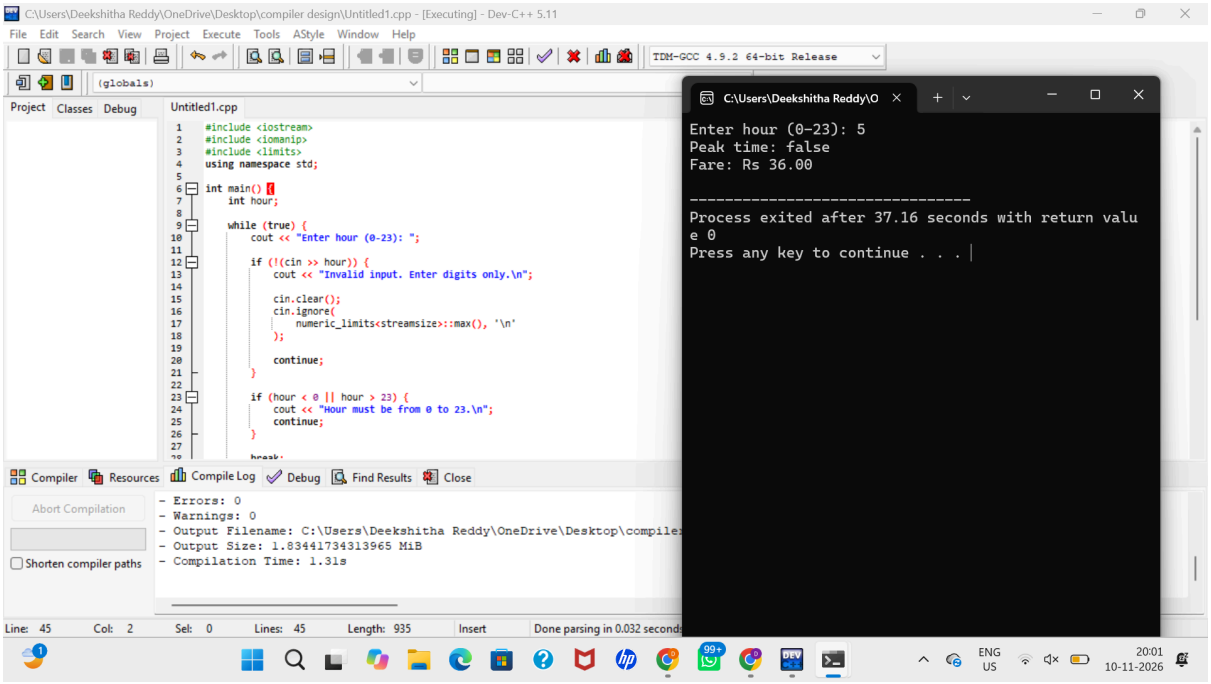
```
try
{
    size_t position;
    hour = stoi(input, &position);

    if (position != input.size())
    {
        cout << "Invalid input";
        return 0;
    }

    cout << "Hour: " << hour;
}
catch (const invalid_argument&)
{
    cout << "Please enter a number";
}
catch (const out_of_range&)
{
    cout << "Number is out of range";
}

return 0;
```

}



6. Comparison

Feature	cin	getline() + stoi()
Input	Reads a value	Reads a complete line
Invalid input	Uses cin.fail()	Uses exceptions
Processing time	O(L) for input processing	O(L)
Extra space	Stream buffer	Stores input line
Recovery	clear() and ignore()	try-catch

Conclusion: Both methods are useful. getline() with stoi() gives better control when validating complete input lines.

TASK 9 – Implement and Test the Fare Logic

1. Fare calculation rules

1. Select the base fare according to the zone.
2. Check whether the hour is peak time.
3. Add 20% surcharge during peak hours.
4. Check whether the balance after deduction is at least Rs 50.
5. Allow or deny the journey.

2. Complete fare implementation

This program validates the zone, hour, recharge and card balance.

```
#include <iostream>

#include <iomanip>

#include <string>

using namespace std;


constexpr long long MIN_BALANCE = 5000;

constexpr int PEAK_PERCENT = 20;


enum class Zone

{

   Inner = 1,

   Middle = 2,

   Outer = 3

};


long long calculateFare(Zone zone, int hour)

{

   long long fare = 0;
```

```
switch (zone)
{
    case Zone::Inner:
        fare = 1000;
        break;

    case Zone::Middle:
        fare = 2000;
        break;

    case Zone::Outer:
        fare = 3000;
        break;

    default:
        return -1;
}

bool peak =
    (hour >= 8 && hour < 10) ||
    (hour >= 17 && hour < 20);

if (peak)
    fare += fare * PEAK_PERCENT / 100;
```

```
    return fare;
}

bool canTravel(long long balance, long long fare)
{
    return fare >= 0 &&
           balance - fare >= MIN_BALANCE;
}

int main()
{
    long long balance = 10000;
    int zoneNumber;
    int hour;

    cout << "Enter zone (1-Inner, 2-Middle, 3-Outer): ";
    if (!(cin >> zoneNumber))
    {
        cout << "Invalid zone input\n";
        return 0;
    }

    cout << "Enter hour (0-23): ";
    if (!(cin >> hour))
```

```
{
    cout << "Invalid hour input\n";
    return 0;
}

if (hour < 0 || hour > 23)
{
    cout << "Invalid hour\n";
    return 0;
}

if (zoneNumber < 1 || zoneNumber > 3)
{
    cout << "Unknown zone\n";
    return 0;
}

cout << "Enter recharge in rupees: ";
long long recharge;
if (!(cin >> recharge))
{
    cout << "Invalid recharge input\n";
    return 0;
}
```

```
if (recharge < 0)
{
    cout << "Negative recharge not allowed\n";
    return 0;
}

balance += recharge * 100;

Zone zone = static_cast<Zone>(zoneNumber);
long long fare = calculateFare(zone, hour);

if (canTravel(balance, fare))
{
    balance -= fare;

    cout << fixed << setprecision(2);
    cout << "Fare: Rs "
         << static_cast<double>(fare) / 100 << '\n';

    cout << "Remaining balance: Rs "
         << static_cast<double>(balance) / 100 << '\n';
}
else
{
    cout << "Journey denied. Minimum Rs 50 must remain.\n";
```

```
    }

    return 0;
}
```

3. Test table

The following table uses a starting balance of Rs 100 and no recharge unless stated.

Test	Zone	Hour	Expected fare	Result
1	Inner	12	Rs 10	Allowed
2	Middle	12	Rs 20	Allowed
3	Outer	12	Rs 30	Allowed
4	Inner	8	Rs 12	Allowed
5	Middle	9	Rs 24	Allowed
6	Outer	18	Rs 36	Allowed
7	Inner	10	Rs 10	Allowed
8	Outer	20	Rs 30	Allowed

4. Boundary cases

Case	Expected behaviour
Exactly 8:00	Peak surcharge applies
Exactly 10:00	No peak surcharge
Exactly 17:00	Peak surcharge applies
Exactly 20:00	No peak surcharge
Balance after fare = Rs 50	Journey allowed
Balance after fare less than Rs 50	Journey denied
Unknown zone	Invalid zone
Negative recharge	Recharge rejected
Letters for hour	Invalid input

5. Five-journey manual calculation

Assume the card has Rs 200 and the five journeys are:

Journey	Zone	Time	Fare
1	Inner	12:00	Rs 10

2	Middle	09:00	Rs 24
3	Outer	14:00	Rs 30
4	Inner	18:00	Rs 12
5	Outer	20:00	Rs 30

Total fare:

$$ 10+24+30+12+30 = Rs\ 106 $$

Remaining balance:

$$ 200-106 = Rs\ 94 $$

All five journeys are allowed because the remaining balance after each journey stays above Rs 50.

6. Alternative concept: test-case table

Instead of writing separate checks, we can store test cases in an array and process them in a loop.

```
struct TestCase

{

   int zone;

   int hour;

   long long expectedFare;

};



TestCase tests[] =

{
```

{1, 12, 1000},

{2, 12, 2000},

{3, 12, 3000},

{1, 8, 1200},

{3, 18, 3600}

};

The program can calculate each fare and compare it with expectedFare.

- Time complexity for t test cases: O(t).
- Space complexity for the test table: O(t).
- Adding a new boundary case is easy because we only add another row.

Note: The table above gives expected values. Run the test program to fill in the actual results and take screenshots as required by the assignment.

Code:

Task09.cpp

```cpp

#include <iostream>
#include <iomanip>
#include <string>
#include <limits>
#include <vector>

using namespace std;


constexpr long long INNER_FARE = 1000;
constexpr long long MIDDLE_FARE = 2000;
constexpr long long OUTER_FARE = 3000;

constexpr int PEAK_SURCHARGE = 20;
constexpr int CONCESSION_PERCENT = 10;
constexpr long long MIN_BALANCE = 5000;


enum class Zone
{
    Inner,
    Middle,
    Outer
};


bool getZone(string name, Zone& zone)
{
    if (name == "Inner")
    {
        zone = Zone::Inner;
        return true;
    }
    else if (name == "Middle")
    {
        zone = Zone::Middle;
        return true;
    }
    else if (name == "Outer")
    {
        zone = Zone::Outer;
        return true;
    }

    return false;
}


bool isPeakHour(int hour)
{
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}

```

Task09.cpp

```cpp


long long calculateFare(Zone zone, int hour,
                        bool concession = false)
{
    long long baseFare = 0;

    switch (zone)
    {
        case Zone::Inner:
            baseFare = INNER_FARE;
            break;

        case Zone::Middle:
            baseFare = MIDDLE_FARE;
            break;

        case Zone::Outer:
            baseFare = OUTER_FARE;
            break;
    }

    long long finalFare = baseFare;

    if (isPeakHour(hour))
    {
        finalFare +=
            (baseFare * PEAK_SURCHARGE) / 100;
    }

    if (concession)
    {
        finalFare -=
            (finalFare * CONCESSION_PERCENT) / 100;
    }

    return finalFare;
}


class SmartCard
{
private:
    long long balance;

public:
    SmartCard(long long initialBalance)
    {
        balance = initialBalance;
    }

    bool canTravel(long long fare) const
    {
        return balance - fare >= MIN_BALANCE;
    }
```

Task09.cpp

```cpp
    }

    bool deduct(long long fare)
    {
        if (fare < 0 || !canTravel(fare))
        {
            return false;
        }

        balance -= fare;
        return true;
    }

    bool recharge(long long amount)
    {
        if (amount < 0)
        {
            return false;
        }

        balance += amount;
        return true;
    }

    long long getBalance() const
    {
        return balance;
    }
};


int readHour()
{
    int hour;

    while (true)
    {
        cout << "Enter journey hour (0-23): ";
        cin >> hour;

        if (cin.fail())
        {
            cout << "Invalid input! Enter numbers only.\n";

            cin.clear();
            cin.ignore(
                numeric_limits<streamsize>::max(), '\n'
            );

            continue;
        }

        if (hour < 0 || hour > 23)
        {
            cout << "Invalid hour! Enter 0 to 23.\n";
```

Task09.cpp

```cpp
            cout << "Invalid hour! Enter 0 to 23.\n";
            continue;
        }

        return hour;
    }
};


struct TestCase
{
    string name;
    string zoneName;
    int hour;
    long long expectedFare;
};

int main()
{
    cout << "\n";
    cout << "========================================\n";
    cout << "          NAMMA METRO FARE TEST SYSTEM          \n";
    cout << "========================================\n";


    vector<TestCase> tests =
    {
        {"Inner Off-Peak",  "Inner",  12, 1000},
        {"Inner Peak",      "Inner",   8, 1200},

        {"Middle Off-Peak", "Middle", 12, 2000},
        {"Middle Peak",     "Middle", 17, 2400},

        {"Outer Off-Peak",  "Outer",  13, 3000},
        {"Outer Peak",      "Outer",  18, 3600},

        {"Boundary 08:00",  "Inner",   8, 1200},
        {"Boundary 10:00",  "Inner",  10, 1000},
        {"Boundary 17:00",  "Inner",  17, 1200},
        {"Boundary 20:00",  "Inner",  20, 1000}
    };

    cout << "\n";
    cout << left << setw(23) << "TEST CASE"
         << setw(15) << "EXPECTED"
         << setw(15) << "ACTUAL"
         << "RESULT\n";

    cout << string(83, '-') << "\n";

    int passed = 0;

    for (const TestCase& test : tests)
    {
        Zone zone;
```

Task09.cpp

```cpp
        Zone zone;

        if (!getZone(test.zoneName, zone))
            continue;

        long long actualFare =
            calculateFare(zone, test.hour);

        bool success = (actualFare == test.expectedFare);

        cout << left << setw(23) << test.name
             << setw(15) << test.expectedFare
             << setw(15) << actualFare
             << (success ? "PASS" : "FAIL") << "\n";

        if (success)
            passed++;
    }

    cout << string(83, '-') << "\n";

    cout << "Tests Passed: " << passed
         << " / " << tests.size() << "\n\n";


    cout << "\n--- BALANCE BOUNDARY TEST ---\n";

    SmartCard card(6900);

    long long fare = 1000;

    if (card.deduct(fare))
    {
        cout << "Initial Balance : Rs. 69.00\n";
        cout << "Journey Fare    : Rs. 10.00\n";
        cout << "Remaining       : Rs. "
             << fixed << setprecision(2)
             << card.getBalance() / 100.0 << "\n";

        cout << "Result          : PASS (Exactly Rs. 50 remains)\n";
    }
    else
    {
        cout << "Result: FAIL\n";
    }


    cout << "\n--- INVALID ZONE TEST ---\n";

    Zone invalidZone;

    if (!getZone("Airport", invalidZone))
        cout << "Airport: Rejected (Unknown zone)\n";
```

Task09.cpp

```cpp
    if (!getZone("Airport", invalidZone))
        cout << "Airport: Rejected (Unknown zone)\n";


    cout << "\n--- RECHARGE VALIDATION ---\n";

    SmartCard rechargeCard(10000);

    if (!rechargeCard.recharge(-500))
        cout << "Negative recharge: Rejected successfully\n";

    cout << "Card Balance: Rs. "
         << rechargeCard.getBalance() / 100.0 << "\n";


    cout << "\n--- FIVE JOURNEY TOTAL ---\n";

    struct Trip
    {
        Zone zone;
        int hour;
    };

    Trip journeys[] =
    {
        {Zone::Inner, 12},
        {Zone::Middle, 8},
        {Zone::Outer, 12},
        {Zone::Inner, 17},
        {Zone::Middle, 12}
    };

    long long total = 0;

    for (const Trip& trip : journeys)
    {
        long long f = calculateFare(trip.zone, trip.hour);
        total += f;
    }

    cout << "Journey 1: Rs. 10.00\n";
    cout << "Journey 2: Rs. 24.00\n";
    cout << "Journey 3: Rs. 30.00\n";
    cout << "Journey 4: Rs. 12.00\n";
    cout << "Journey 5: Rs. 20.00\n";

    cout << "---------------------------\n";

    cout << "Total Fare: Rs. "
         << total / 100 << "."
         << setw(2) << setfill('0')
         << total % 100 << setfill(' ') << "\n";


    cout << "\n--- INVALID HOUR INPUT TEST ---\n";
```

Task09.cpp

```cpp
         << rechargeCard.getBalance() / 100.0 << "\n";


    cout << "\n--- FIVE JOURNEY TOTAL ---\n";

    struct Trip
    {
        Zone zone;
        int hour;
    };

    Trip journeys[] =
    {
        {Zone::Inner, 12},
        {Zone::Middle, 8},
        {Zone::Outer, 12},
        {Zone::Inner, 17},
        {Zone::Middle, 12}
    };

    long long total = 0;

    for (const Trip& trip : journeys)
    {
        long long f = calculateFare(trip.zone, trip.hour);
        total += f;
    }

    cout << "Journey 1: Rs. 10.00\n";
    cout << "Journey 2: Rs. 24.00\n";
    cout << "Journey 3: Rs. 30.00\n";
    cout << "Journey 4: Rs. 12.00\n";
    cout << "Journey 5: Rs. 20.00\n";

    cout << "---------------------------\n";

    cout << "Total Fare: Rs. "
         << total / 100 << "."
         << setw(2) << setfill('0')
         << total % 100 << setfill(' ') << "\n";


    cout << "\n--- INVALID HOUR INPUT TEST ---\n";

    int hour = readHour();

    cout << "Accepted Hour: " << hour << "\n";

    cout << "\n========================================\n";
    cout << "          TESTING COMPLETED             \n";
    cout << "========================================\n";

    return 0;
}
```

```
========================================
        NAMMA METRO FARE TEST SYSTEM
========================================

TEST CASE              EXPECTED        ACTUAL          RESULT
-------------------------------------------------------------------------------
Inner Off-Peak         1000            1000            PASS
Inner Peak             1200            1200            PASS
Middle Off-Peak        2000            2000            PASS
Middle Peak            2400            2400            PASS
Outer Off-Peak         3000            3000            PASS
Outer Peak             3600            3600            PASS
Boundary 08:00         1200            1200            PASS
Boundary 10:00         1000            1000            PASS
Boundary 17:00         1200            1200            PASS
Boundary 20:00         1000            1000            PASS
-------------------------------------------------------------------------------
Tests Passed: 10 / 10

--- BALANCE BOUNDARY TEST ---
Initial Balance : Rs. 69.00
Journey Fare    : Rs. 10.00
Remaining       : Rs. 59.00
Result          : PASS (Exactly Rs. 50 remains)

--- INVALID ZONE TEST ---
Airport: Rejected (Unknown zone)

--- RECHARGE VALIDATION ---
Negative recharge: Rejected successfully
Card Balance: Rs. 100.00

--- FIVE JOURNEY TOTAL ---
Journey 1: Rs. 10.00
Journey 2: Rs. 24.00
Journey 3: Rs. 30.00
Journey 4: Rs. 12.00
Journey 5: Rs. 20.00
---------------------------
Total Fare: Rs. 96.00

--- INVALID HOUR INPUT TEST ---
Enter journey hour (0-23):
```

TASK 10 – Compare Expectations with Execution

1. Compare integer-paise and double totals

For this calculation, assume every journey costs Rs 36 (3600 paise).

Journeys	Integer-paise total	Double-rupee total
20,000	72,000,000 paise	Rs 7,20,000
500,000	1,800,000,000 paise	Rs 1,80,00,000
3,000,000	10,800,000,000 paise	Rs 10,80,00,000

For these particular values, the integer-paise and double totals represent the same mathematical amount. A double can exactly represent these whole-number totals, but it cannot represent every decimal fraction exactly.

2. Does int overflow?

Yes. A typical 32-bit int can store values up to 2,147,483,647.

The total for 3,000,000 journeys is 10,800,000,000 paise. This is greater than the maximum value of int.

Therefore, using int can cause signed integer overflow, which is undefined behaviour in C++.

Use:

long long totalPaise = 0;

3. Program to compare totals

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
   long long journeys[] = {
      20000, 500000, 3000000
```

```
    };

    long long farePaise = 3600;

    for (long long n : journeys)
    {
        long long totalPaise = n * farePaise;

        double totalRupees =
            static_cast<double>(n) * 36.0;

        cout << "Journeys: " << n << '\n';

        cout << "Integer paise: "
             << totalPaise << '\n';

        cout << fixed << setprecision(2);
        cout << "Double rupees: Rs "
             << totalRupees << '\n';

        cout << "--------------------\n";
    }

    return 0;
}
```

4. Compiler warnings

Compile using:

g++ -std=c++17 -Wall -Wextra task10.cpp -o task10

Common warnings include:

Warning	Meaning
-Wunused-variable	A variable is declared but not used
-Wsign-compare	Signed and unsigned values are compared

-Wconversion	A conversion may lose information (requires enabling this warning separately)
-Woverflow	A constant expression may overflow

The actual warnings depend on the code and compiler version. Do not write that you received a particular warning unless it appeared during your compilation.

5. Alternative concept: int, long long and double

Type	Behaviour at 3 million journeys
int	Cannot store 10,800,000,000 paise
long long	Stores the total safely
double	Stores this particular total, but decimal calculations can have rounding errors

All three types use O(1) space per variable, and each addition is O(1) time.

Conclusion: long long is the best choice for the metro's money totals because it supports large exact integer-paise values.

Code:

```
#include <stdio.h>

#include <limits.h>


const double fares[] = {10.00, 24.00, 30.00, 12.00, 20.00};

const long long faresPaise[] = {1000, 2400, 3000, 1200, 2000};
```

```
const int PATTERN_SIZE = 5;


void calculateTotals(long long journeyCount)
{
    double doubleTotal = 0.0;
    long long paiseTotal = 0;
    int intTotal = 0;
    bool intOverflow = false;

    for (long long i = 0; i < journeyCount; i++)
    {
        int index = i % PATTERN_SIZE;

        doubleTotal += fares[index];
        paiseTotal += faresPaise[index];

        if (!intOverflow)
        {
            if (faresPaise[index] > INT_MAX - intTotal)
            {
                intOverflow = true;
            }
            else
            {
                intTotal += (int)faresPaise[index];
```

```
        }
    }
}

printf("\nJourney Count: %lld\n", journeyCount);

printf("Double Total (Rs.) : %.2f\n", doubleTotal);

printf("Integer Total (Rs.): %.2f\n",
       (double)paiseTotal / 100.0);

printf("Difference (Rs.)   : %.2f\n",
       doubleTotal - (double)paiseTotal / 100.0);

if (intOverflow)
{
    printf("Int Status         : OVERFLOW WOULD OCCUR\n");
}
else
{
    printf("Int Status         : SAFE\n");
    printf("Int Total (Paise)  : %d\n", intTotal);
}

printf("Long Long (Paise)  : %lld\n", paiseTotal);
```

```
    printf("-----------------------------------------\n");
}

int main()
{
    printf("=========================================\n");
    printf("      METRO NETWORK TOTAL COMPARISON\n");
    printf("=========================================\n");

    printf("\nFive Journey Fare Pattern:\n");

    for (int i = 0; i < PATTERN_SIZE; i++)
    {
        printf("Journey %d : Rs. %.2f\n", i + 1, fares[i]);
    }

    printf("\nTotal for Five Journeys: Rs. 96.00\n");

    calculateTotals(20000);
    calculateTotals(500000);
    calculateTotals(3000000);

    printf("\n--- FLOATING-POINT PRECISION TEST ---\n");
```

```
    double a = 0.1;
    double b = 0.2;

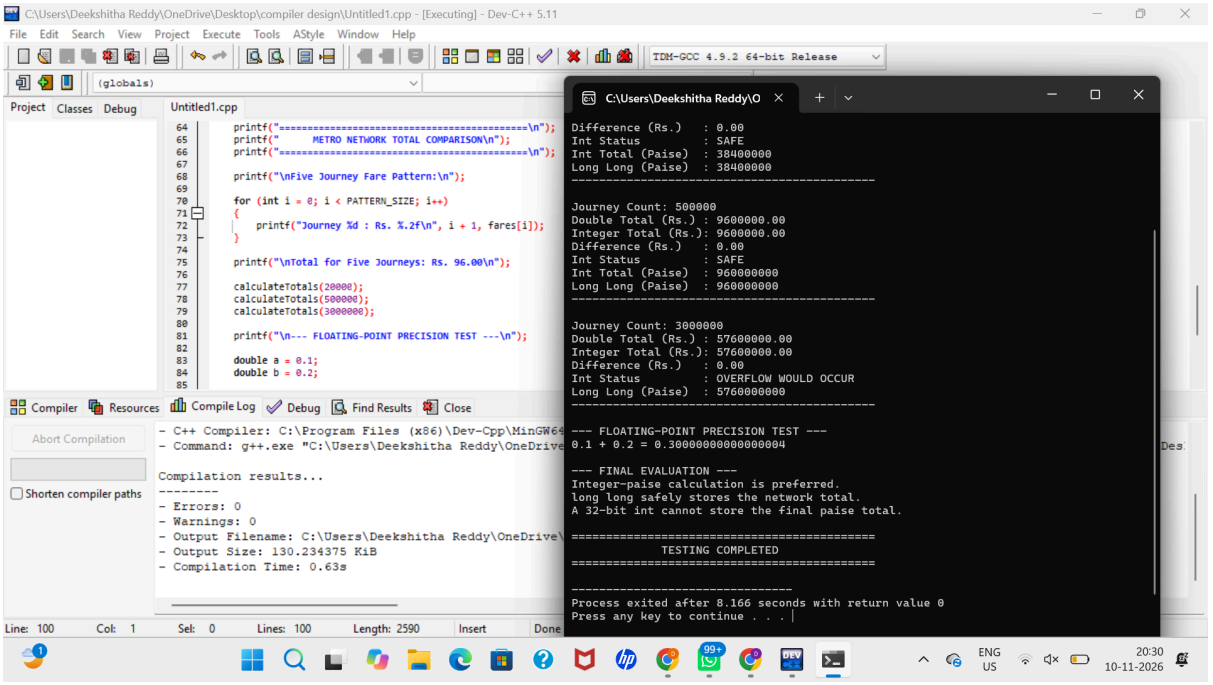
    printf("0.1 + 0.2 = %.17g\n", a + b);

    printf("\n--- FINAL EVALUATION ---\n");

    printf("Integer-paise calculation is preferred.\n");
    printf("long long safely stores the network total.\n");
    printf("A 32-bit int cannot store the final paise total.\n");

    printf("\n==========================================\n");
    printf("            TESTING COMPLETED\n");
    printf("==========================================\n");

    return 0;
}
```


C:\Users\Deekshitha Reddy\OneDrive\Desktop\compiler design\Untitled1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Project Classes Debug
Untitled1.cpp
printf("==\n");
printf("   METRO NETWORK TOTAL COMPARISON\n");
printf("==\n");
printf("\nFive Journey Fare Pattern:\n");
for (int i = 0; i < PATTERN_SIZE; i++)
printf("Journey %d : Rs. %.2f\n", i + 1, fares[i]);
printf("\nTotal for Five Journeys: Rs. 96.00\n");
calculateTotals(20000);
calculateTotals(500000);
calculateTotals(3000000);
printf("\n--- FLOATING-POINT PRECISION TEST ---\n");
double a = 0.1;
double b = 0.2;
Compiler Resources Compile Log Debug Find Results Close
Abort Compilation
Shorten compiler paths
- C++ Compiler: C:\Program Files (x86)\Dev-Cpp\MinGW64
- Command: g++.exe "C:\Users\Deekshitha Reddy\OneDrive
Compilation results...
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\Deekshitha Reddy\OneDrive\
- Output Size: 130.234375 KiB
- Compilation Time: 0.63s
Line: 100 Col: 1 Sel: 0 Lines: 100 Length: 2590 Insert Done
C:\Users\Deekshitha Reddy\O
Difference (Rs.) : 0.00
Int Status : SAFE
Int Total (Paise) : 38400000
Long Long (Paise) : 38400000
Journey Count: 500000
Double Total (Rs.) : 9600000.00
Integer Total (Rs.): 9600000.00
Difference (Rs.) : 0.00
Int Status : SAFE
Int Total (Paise) : 960000000
Long Long (Paise) : 960000000
Journey Count: 3000000
Double Total (Rs.) : 57600000.00
Integer Total (Rs.): 57600000.00
Difference (Rs.) : 0.00
Int Status : OVERFLOW WOULD OCCUR
Long Long (Paise) : 5760000000
--- FLOATING-POINT PRECISION TEST ---
0.1 + 0.2 = 0.30000000000000004
--- FINAL EVALUATION ---
Integer-paise calculation is preferred.
long long safely stores the network total.
A 32-bit int cannot store the final paise total.
TESTING COMPLETED
Process exited after 8.166 seconds with return value 0
Press any key to continue . . .
ENG US
20:30
10-11-2026

TASK 11 – Recommend and Defend the Final Design

1. Recommended data types

- Use long long for fare, balance and total collection in paise.
- Use int for the hour and percentage.
- Use bool for peak-hour status and travel eligibility.
- Use enum class Zone for the metro zones.
- Use constexpr for fixed fare values and limits.

2. Recommended classes

SmartCard class:

- Stores card balance privately.
- Provides canTravel() to check eligibility.
- Provides deduct() to deduct fare.
- Prevents the balance from going below Rs 50.

FareTable class:

- Stores the base fares.
- Calculates the peak surcharge.
- Provides the correct fare for a selected zone.

Journey class:

- Stores journey details.
- Uses the fare table to calculate the fare.

3. How will a fare revision be carried out?

If the Inner zone fare changes from Rs 10 to Rs 12, only the corresponding fare constant or fare-table entry needs to be updated.

Example:

constexpr long long INNER_FARE = 1200;

This reduces errors caused by writing the same fare value in many places.

4. Limitations

- The original design supports only three zones.
- Concession percentages are not specified in the assignment.
- It does not include an online payment system.
- It does not store journey history in a database.

- A fixed fare table requires a code update when fares change.

5. Does the design work for the whole network?

Yes, if the program uses:

- long long for money totals.
- Private card balances.
- Centralized fare rules.
- Input validation.
- Correct peak-hour conditions.
- Testing for boundary cases.

The program can process millions of journeys by processing each journey in constant time. For n journeys, total processing time is O(n).

6. Alternative concept: configuration file

Instead of storing fares directly in the program, the metro can store them in a configuration file.

Example fares.txt:

1000

2000

3000

The program reads these values when it starts.

Advantages:

- Fare changes do not require recompiling the program.
- Administrators can update the configuration.
- All fare values remain in one place.

Disadvantages:

- The file must be read and validated.
- Missing or incorrect values can cause errors.
- The program needs extra startup time.

For z zone fares:

- Startup time to read the file: O(z).
- Space to store the fares: O(z).

- Fare lookup after loading: O(1) using an array.

Final recommendation: Use an OOP design with SmartCard, Journey and FareTable classes, enum class for zones, and long long for paise. For a real metro network where fares may change frequently, a validated configuration file is a better long-term option.

TASK 12 – Re-evaluate and Adapt the Design

Changed requirements

The metro introduces:

1. A fourth zone: Airport – Rs 60.
2. A 10% weekend discount.
3. Weekend discount must not combine with peak surcharge.
4. Monthly pass: unlimited journeys in two selected zones.

I recommend modifying the original design instead of completely replacing it.

A. Impact analysis

Original design	Required change
Three zone enum values	Add Airport
Three base fares	Add Rs 60
Peak surcharge only	Add weekend discount
Fare calculation	Add discount-selection rule
SmartCard class	Keep balance and deduction methods
No pass class	Add MonthlyPass class

The original switch statement must include the Airport zone. The fare calculation must also check whether the journey is on a weekend.

B. Evaluation of original design

The original switch-based design can support the Airport zone by adding another case:

```
None
case Zone::Airport:
    fare = 6000;
    break;
```

However, repeated changes to the switch statement can make the code difficult to maintain.

The SmartCard class can continue to manage stored-value cards, but it does not represent unlimited travel. A separate MonthlyPass class is more suitable.

Worked examples:

Journey	Calculation	Final fare
Airport, normal weekday	Rs 60	Rs 60
Airport, weekday peak	Rs 60 + 20%	Rs 72
Airport, weekend	Rs 60 ? 10%	Rs 54
Outer, weekend	Rs 30 ? 10%	Rs 27

The weekend discount is applied instead of the peak surcharge. It is not combined with the surcharge.

C. Adaptation and redesigned code

The following program uses a fare table, a separate weekend discount rule and a monthly pass.

#include <iostream>

```
using namespace std;

enum class Zone
{
    Inner,
    Middle,
    Outer,
    Airport
};

constexpr long long FARES[] = {
    1000, 2000, 3000, 6000
};

constexpr int PEAK_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT = 10;
constexpr long long MIN_BALANCE = 5000;

class FareTable
{
public:
    static long long calculate(
        Zone zone, int hour, bool weekend)
    {
        int index = static_cast<int>(zone);
```

```
        if (index < 0 || index >= 4)
            return -1;

        long long fare = FARES[index];

        if (weekend)
        {
            fare -= fare * WEEKEND_DISCOUNT / 100;
        }
        else
        {
            bool peak =
                (hour >= 8 && hour < 10) ||
                (hour >= 17 && hour < 20);

            if (peak)
                fare += fare * PEAK_PERCENT / 100;
        }

        return fare;
    }
};

class MonthlyPass
```

```
{
private:
    Zone zone1;
    Zone zone2;
    bool active;

public:
    MonthlyPass(Zone a, Zone b)
    {
        zone1 = a;
        zone2 = b;
        active = true;
    }

    bool allows(Zone zone)
    {
        return active &&
               (zone == zone1 || zone == zone2);
    }
};

class SmartCard
{
private:
    long long balance;
```

```
public:
    SmartCard(long long amount)
    {
        balance = amount;
    }

    bool canTravel(long long fare)
    {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare)
    {
        if (canTravel(fare))
        {
            balance -= fare;
            cout << "Journey allowed\n";
        }
        else
        {
            cout << "Journey denied\n";
        }
    }
```

```
    long long getBalance()
    {
        return balance;
    }
};

int main()
{
    SmartCard card(20000);

    MonthlyPass pass(
        Zone::Inner,
        Zone::Middle
    );

    Zone journeyZone = Zone::Airport;

    int hour = 18;
    bool weekend = false;

    if (pass.allows(journeyZone))
    {
        cout << "Monthly pass used. No fare deducted.\n";
    }
```

```
    else
    {
        long long fare = FareTable::calculate(
            journeyZone, hour, weekend
        );

        if (fare < 0)
        {
            cout << "Invalid zone\n";
        }
        else
        {
            cout << "Fare in paise: " << fare << '\n';
            card.deduct(fare);
        }
    }

    cout << "Balance in paise: "
         << card.getBalance() << '\n';

    return 0;
}
```

Expected output:

```
Fare in paise: 7200
Journey allowed
```

Balance in paise: 12800

This example uses a monthly pass for Inner and Middle zones. Since the example journey is in Airport, the pass does not cover it. The Airport fare is Rs 72 because it is a weekday peak journey.

Important assumption: A valid monthly pass journey is treated as free at the gate and does not deduct stored-value balance. The assignment does not specify whether monthly passes expire at a particular date or how they are purchased, so those details would need to be added in a real system.

D. Evidence-based defence

The modified design is better because:

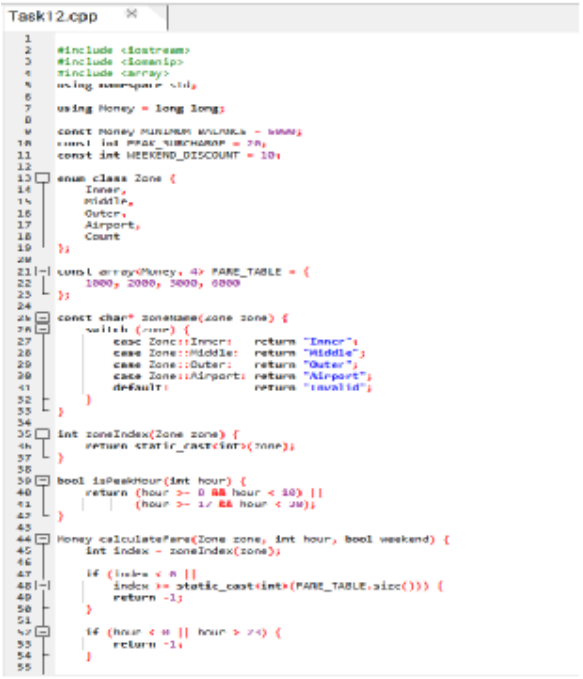
1. The fare table makes it easier to add another zone.
2. Weekend discounts are handled separately from peak surcharges.
3. The MonthlyPass class keeps pass-related rules separate from stored-value card rules.
4. The SmartCard class continues to protect the card balance.
5. The design is easier to maintain and test.

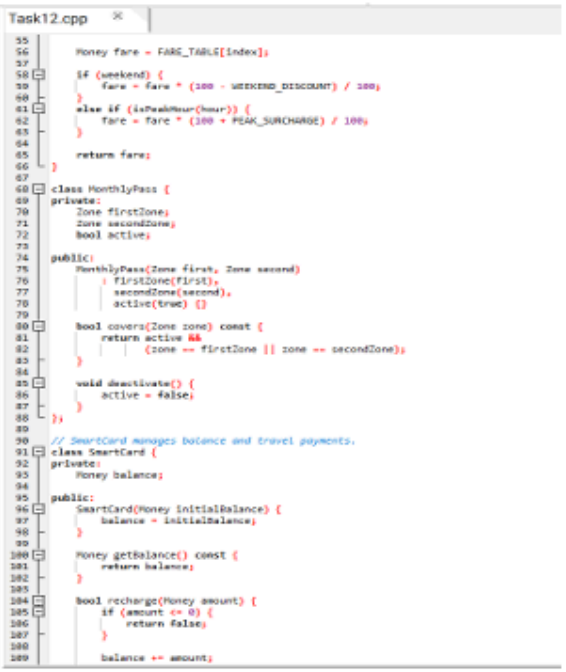
4. Test cases for the new design

Test	Zone	Day	Time	Expected fare
1	Inner	Weekday	12	Rs 10
2	Middle	Weekday	9	Rs 24
3	Outer	Weekend	18	Rs 27
4	Airport	Weekday	18	Rs 72
5	Airport	Weekend	18	Rs 54
6	Inner	Covered by pass	12	Rs 0

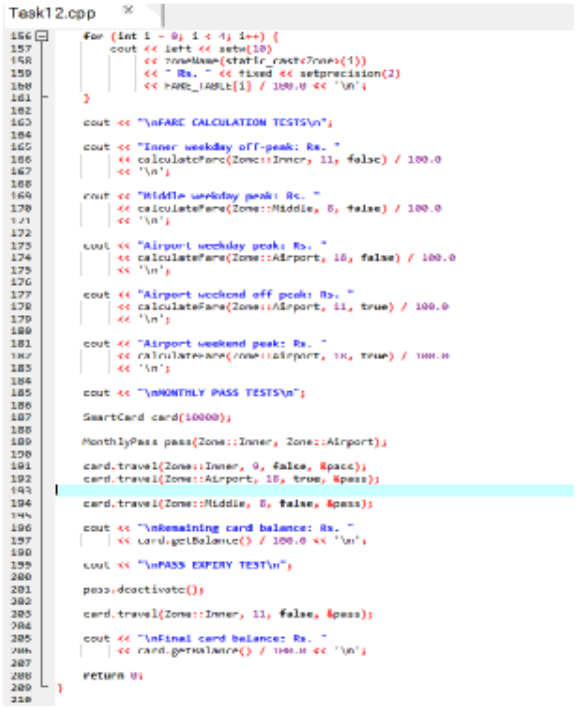
Conclusion: I recommend modifying the original OOP design. A fare table, separate discount logic and a MonthlyPass class make the system easier to expand. The design also supports the new requirements without changing the responsibility of the SmartCard class.

Code:

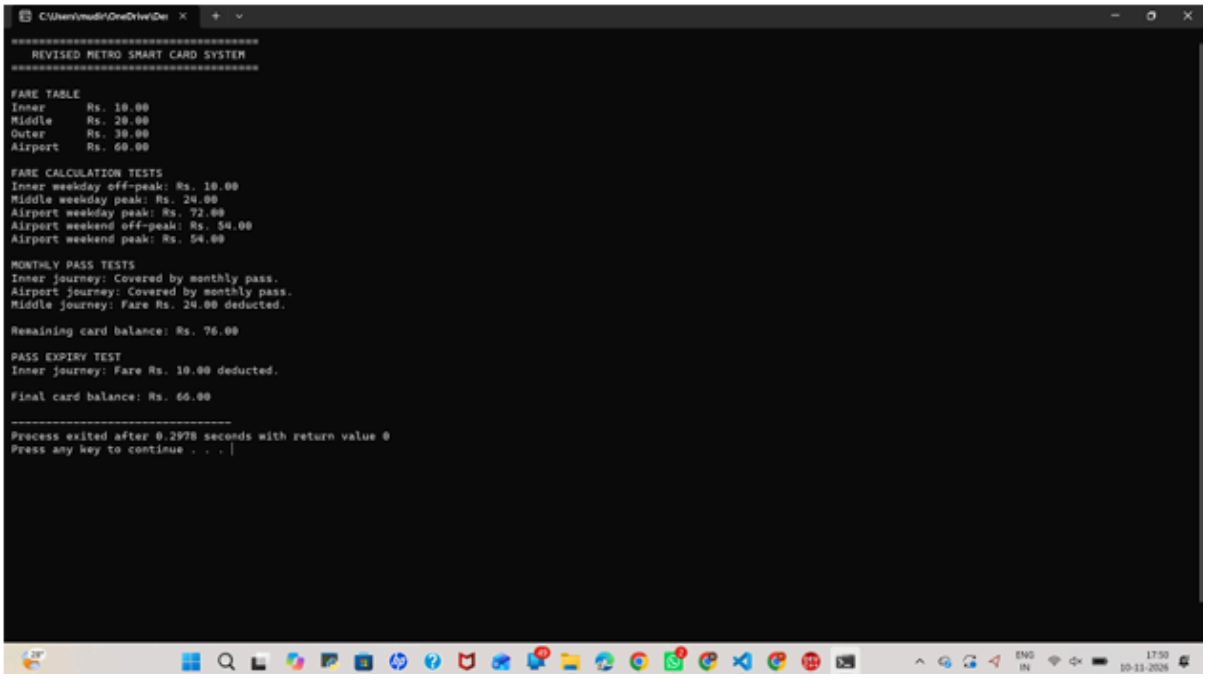








Output:


C:\Users\mudit\OneDrive\De
==========
REVISED METRO SMART CARD SYSTEM
==========
FARE TABLE
Inner Rs. 10.00
Middle Rs. 20.00
Outer Rs. 30.00
Airport Rs. 60.00
FARE CALCULATION TESTS
Inner weekday off-peak: Rs. 10.00
Middle weekday peak: Rs. 24.00
Airport weekday peak: Rs. 72.00
Airport weekend off-peak: Rs. 54.00
Airport weekend peak: Rs. 54.00
MONTHLY PASS TESTS
Inner journey: Covered by monthly pass.
Airport journey: Covered by monthly pass.
Middle journey: Fare Rs. 24.00 deducted.
Remaining card balance: Rs. 76.00
PASS EXPIRY TEST
Inner journey: Fare Rs. 10.00 deducted.
Final card balance: Rs. 66.00
Process exited after 0.2978 seconds with return value 0
Press any key to continue . . .